

## MODULE 2 — Multilayer Perceptrons (Simplified Study Material)

### MODULE-2

**Multilayer Perceptrons:** Introduction, Batch Learning and On-Line Learning, The Back-Propagation Algorithm, XOR Problem, Heuristics for Making the Back-Propagation Algorithm Perform Better, Back Propagation and Differentiation.]

## 1. Introduction to Multilayer Perceptrons (MLP)

A **Multilayer Perceptron (MLP)** is a neural network with:

1. **One or more hidden layers**
2. **Nonlinear activation functions** that are differentiable
3. **High connectivity** through synaptic weights

Why MLPs are important:

- A single perceptron can only classify **linearly separable** patterns.
- MLP overcomes these limitations and can learn **complex, nonlinear mappings**.

### Key Features of MLP

- Each neuron uses a **differentiable nonlinear activation function**.
- There are **hidden layers**, which are not directly visible from input/output.
- Learning is performed using weight adjustments.

### Training MLP: Backpropagation

Training happens in **two phases**:

1. **Forward pass** – compute outputs layer by layer.
2. **Backward pass** – compute errors and adjust weights.

Backpropagation became widely popular after 1985.

### Preliminary (not for exam, only for your understanding)

#### Preliminary: How MLP Learns from Data (Simple Introduction)

Before understanding **Batch Learning** and **Online Learning**, students must know *how an MLP learns*.

An MLP learns through **supervised learning**, where we give:

- **Input**  $\rightarrow x(n)$
- **Desired/Target Output**  $\rightarrow d(n)$

The goal of learning is to adjust the **weights** of the network so that the **actual output** becomes **closer** to the desired output.

#### How Errors Are Calculated

For every input  $x(n)$ , the network produces an output  $y_j(n)$ .

The difference between what the network gives and what we expect is the **error**:

$$e_j(n) = d_j(n) - y_j(n)$$

If the error is large  $\rightarrow$  network is performing poorly.

If the error is small  $\rightarrow$  network is learning well.

#### Measuring How Much Error the Network Has

To measure error for the entire network at step  $n$ :  $e(n) = \frac{1}{2} \sum e_j(n)^2$

To measure error across all **N** training examples:  $e_{av}(N) = \frac{1}{N} \sum_{n=1}^N e(n)$

This tells us **how well the MLP is learning overall**.

#### Why Do We Need Learning Methods?

Learning methods decide **when** and **how often** to update weights.

Two common methods:

1. **Batch Learning**
2. **Online (Stochastic) Learning**

These methods control how fast the MLP learns, how well it generalizes, and whether it gets stuck during training.

## 2. Batch Learning and Online Learning

MLP learning uses training examples  $(x(n), d(n))$  WHERE  $x(n)$  :  $n^{\text{th}}$  input,  $d(n)$ :  $n^{\text{th}}$  actual output

We define:

- Output of neuron  $j$ :  $y_j(n)$
- Error:  $e_j(n) = d_j(n) - y_j(n)$
- Instantaneous error energy:  $e(n) = \frac{1}{2} \sum e_j^2(n)$
- Average error energy (Empirical risk):

$$e_{av}(N) = \frac{1}{N} \sum_{n=1}^N e(n)$$

### 2.1 Batch Learning

- Weight updates occur after all  $N$  samples in one epoch.
- Uses average error  $e_{av}$ .
- Pros:
  - More accurate gradient.
  - Easily parallelized.
- Cons:
  - Needs high memory.
  - Slower for large datasets.

Best used for nonlinear regression problems.

---

### 2.2 Online Learning (Stochastic Learning)

- Weight updates occur after every training example.
- Pros:
  - Requires less storage.
  - Avoids getting stuck in local minima due to randomness.
  - Tracks small changes when data is non-stationary.
- Cons:
  - Harder to parallelize.

Hence widely used for pattern classification tasks.

## 3. The Back-Propagation Algorithm

Text source: Section 4.4, pages 129–140

Backpropagation adjusts weights using **gradient descent**. Back-propagation is a **learning algorithm** used to train **multilayer perceptrons (MLPs)**. It is basically **gradient descent** on the error surface, efficiently implemented using the **chain rule of calculus**.

To describe this algorithm, consider Fig. 4.3, which depicts neuron  $j$  being fed by a set of function signals produced by a layer of neurons to its left. The induced local field  $v_j(n)$  produced at the input of the activation function associated with neuron  $j$

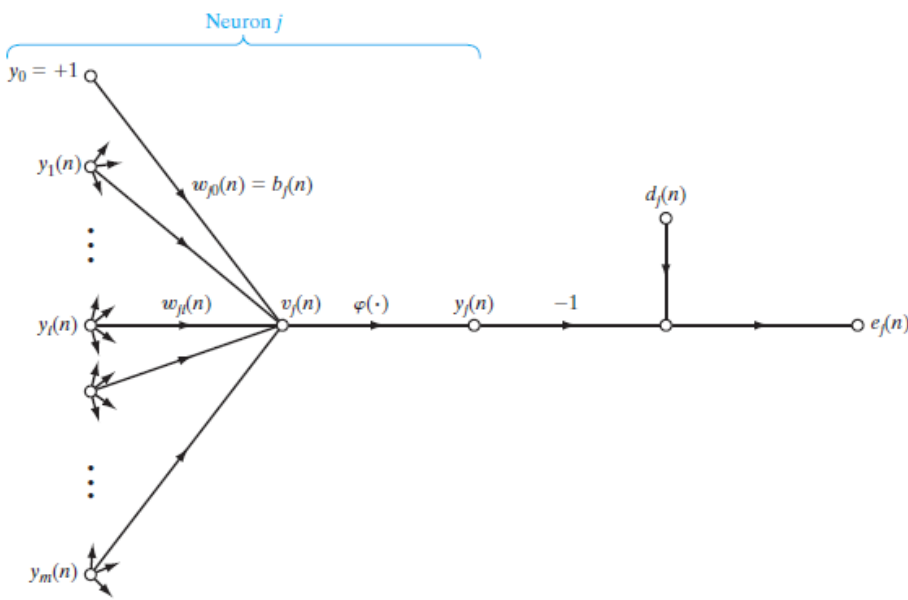


FIGURE 4.3 Signal-flow graph highlighting the details of output neuron  $j$ .

### 3.1 1. Neuron model and induced local field

Consider **neuron  $j$**  in some layer of the network (for now, think of it in the output layer). It receives inputs from a set of neurons to its left (previous layer).

The **induced local field** of neuron  $j$  at iteration  $n$  is

$$v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad (4.6)$$

- $m$ : total number of inputs **excluding** the bias
- $y_i(n)$ : input signal from neuron  $i$  (or from input node if first hidden layer)
- $w_{ji}(n)$ : synaptic weight from neuron  $i$  to neuron  $j$
- $i = 0$  corresponds to the **bias input**  $y_0 = +1$ , and  $w_{j0}(n) = b_j(n)$  is the bias weight.

The **output (function signal)** of neuron  $j$  is obtained by passing the induced local field through the activation function  $\phi_j(\cdot)$ :

$$y_j(n) = \phi_j(v_j(n)) \quad (4.7)$$

### Objective: minimize instantaneous error energy

The total instantaneous error energy of the network at iteration  $n$  is

$$e(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n) \quad (4.4)$$

where

- $C$  = set of output neurons
- $e_j(n) = d_j(n) - y_j(n)$  is the error of neuron  $j$ . \tag{4.2}

We want to **change the weights** so that  $e(n)$  decreases.

## Gradient descent on weights

For weight  $w_{ji}(n)$ , the gradient of the error is

$$\frac{\partial e(n)}{\partial w_{ji}(n)} \quad (4.8)$$

Using the **chain rule**, we expand this derivative as:

$$\frac{\partial e(n)}{\partial w_{ji}(n)} = \frac{\partial e(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)} \quad (4.8)$$

Now, compute each factor one by one:

1. From  $e(n) = \frac{1}{2} \sum e_j^2(n)$ :

$$\frac{\partial e(n)}{\partial e_j(n)} = e_j(n) \quad (4.9)$$

2. From  $e_j(n) = d_j(n) - y_j(n)$ :

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1 \quad (4.10)$$

3. From  $y_j(n) = \phi_j(v_j(n))$ :

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \phi'_j(v_j(n)) \quad (4.11)$$

4. From  $v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n)$ :

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n) \quad (4.12)$$

Putting them back into (4.8):

$$\frac{\partial e(n)}{\partial w_{ji}(n)} = -e_j(n) \phi'_j(v_j(n)) y_i(n) \quad (4.13)$$

## 3.2 Weight Adjustment Rule (Delta Rule)

The delta rule for updating weight  $w_{ji}$  is  $\Delta w_{ji}(n) = -\eta \frac{\partial e(n)}{\partial w_{ji}(n)}$  where  $\eta$  is the **learning-rate parameter**.

Substituting (4.13) into (4.14): After applying chain rule:

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

where

- $\delta_j(n)$  is the **local gradient**

$$\text{From the calculations above, } \delta_j(n) = e_j(n) \phi'_j(v_j(n)) \quad (4.16)$$

So, **weight change = learning rate × local gradient × input.**

### 3.3 Local Gradient Calculation

#### Case 1: Output Neuron

For an output neuron, we already have:

$$\delta_j(n) = e_j(n) \phi'_j(v_j(n))$$

This is straightforward because the **desired response**  $d_j(n)$  is available.

#### Case 2: Hidden Neuron

Hidden neurons **do not have target outputs** directly. So we must express their error in terms of the errors of neurons in the next layer. We redefine the local gradient for hidden neuron  $j$  as

$$\delta_j(n) = -\frac{\partial e(n)}{\partial y_j(n)} \phi'_j(v_j(n)) \quad (4.17)$$

To compute  $\frac{\partial e(n)}{\partial y_j(n)}$ , consider the error energy written with index  $k$  for output neurons:

$$e(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n) \quad (4.18)$$

Differentiating w.r.t.  $y_j(n)$ :

$$\frac{\partial e(n)}{\partial y_j(n)} = \sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j(n)} \quad (4.19)$$

Again, apply chain rule through neuron  $k$ :

$$\frac{\partial e_k(n)}{\partial y_j(n)} = \frac{\partial e_k(n)}{\partial v_k(n)} \cdot \frac{\partial v_k(n)}{\partial y_j(n)} \quad (4.20)$$

For output neuron  $k$ :

$$e_k(n) = d_k(n) - \phi_k(v_k(n)) \Rightarrow \frac{\partial e_k(n)}{\partial v_k(n)} = -\phi'_k(v_k(n)) \quad (4.21-4.22)$$

The induced local field of neuron  $k$  is

$$v_k(n) = \sum_{j=0}^m w_{kj}(n) y_j(n) \quad (4.23)$$

So

$$\frac{\partial v_k(n)}{\partial y_j(n)} = w_{kj}(n) \quad (4.24)$$

Now

$$\begin{aligned} \frac{\partial e(n)}{\partial y_j(n)} &= \sum_k e_k(n) [-\phi'_k(v_k(n))] w_{kj}(n) \\ &= - \sum_k \delta_k(n) w_{kj}(n) \end{aligned} \quad (4.25)$$

because for neuron  $k$  we have  $\delta_k(n) = e_k(n) \phi'_k(v_k(n))$ .

Substitute this into (4.17):

$$\delta_j(n) = \phi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n), (\text{hidden neuron}) \quad (4.26)$$

This is the **back-propagation formula**: the error gradient at a hidden neuron is the **weighted sum of gradients in the next layer**, multiplied by the derivative of its activation function.

### Compact form of the weight update

From (4.15), the update for weight from neuron  $i$  to neuron  $j$  is

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n) \quad (4.27)$$

Interpretation:

- $\eta$ : learning-rate parameter
- $\delta_j(n)$ : local gradient (how much neuron  $j$  is responsible for the error)
- $y_i(n)$ : input signal to neuron  $j$

---

## 3.4 Two Passes of BP (The Two Passes of Computation)

**Forward Pass**: Compute all outputs  $y_j(n)$  layer by layer.

For neuron  $j$ :  $y_j(n) = \phi(v_j(n)) \quad (4.28)$

With  $v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad (4.29)$

Special cases:

- If neuron  $j$  is in the **first hidden layer**:  $y_i(n) = x_i(n) \quad (4.30)$

- If neuron  $j$  is in the **output layer**:  $y_j(n) = o_j(n) \quad (4.31)$

where  $o_j(n)$  is the  $j$ -th component of the network output.

Error of output neuron  $j$ :  $e_j(n) = d_j(n) - o_j(n) \quad (4.45)$

### Backward Pass

#### Backward pass and generalized delta rule

We compute local gradients:

For output layer  $L$ :

$$\delta_j^{(L)}(n) = e_j^{(L)}(n) \phi_j'(v_j^{(L)}(n)) \quad (4.46, \text{ first line})$$

For hidden layer  $l$ :

$$\delta_j^{(l)}(n) = \phi_j'(v_j^{(l)}(n)) \sum_k \delta_k^{(l+1)}(n) w_{kj}^{(l+1)}(n) \quad (4.46, \text{ second line})$$

Then we update weights with **momentum** (generalized delta rule):

$$w_{ji}^{(l)}(n+1) = w_{ji}^{(l)}(n) + \beta [\Delta w_{ji}^{(l)}(n-1)] + \eta \delta_j^{(l)}(n) y_i^{(l-1)}(n) \quad (4.47)$$

- $\eta$ : learning-rate parameter
- $\beta$ : momentum constant
- $\Delta w_{ji}^{(l)}(n-1)$ : previous weight change.

---

### 3.5 Activation functions and their derivatives

**(a) Logistic Sigmoid**  $\phi(v) = \frac{1}{1+e^{-av}}, \phi'(v) = a y(1-y)$

In detail:

*Derivative:*

$$\phi_j'(v_j(n)) = a y_j(n) [1 - y_j(n)] \quad (4.33-4.34)$$

*Local gradient for **output neuron** using logistic:*

$$\delta_j(n) = [d_j(n) - o_j(n)] o_j(n) [1 - o_j(n)] \quad (4.35)$$

*Local gradient for **hidden neuron** using logistic:*

$$\delta_j(n) = a y_j(n) [1 - y_j(n)] \sum_k \delta_k(n) w_{kj}(n) \quad (4.36)$$

**(b) Hyperbolic Tangent**  $\phi(v) = a \tanh(bv), \phi'(v) = ab(1 - \tanh^2(bv))$

In detail:

*Derivative:*  $\phi_j'(v_j(n)) = ab \operatorname{sech}^2(bv_j(n)) = \frac{b}{a} [a - y_j(n)][a + y_j(n)] \quad (4.38)$

*Local gradient for **output neuron** with tanh:*

$$\delta_j(n) = \frac{b}{a} [d_j(n) - o_j(n)][a - o_j(n)][a + o_j(n)] \quad (4.39)$$

*Local gradient for **hidden neuron** with tanh:*  $\delta_j(n) = \frac{b}{a} [a - y_j(n)][a + y_j(n)] \sum_k \delta_k(n) w_{kj}(n) \quad (4.40)$

---

### 3.6 Momentum Term

To speed learning:  $\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n)$

Momentum helps:

- Faster convergence in downhill directions.
- Reduces oscillations.

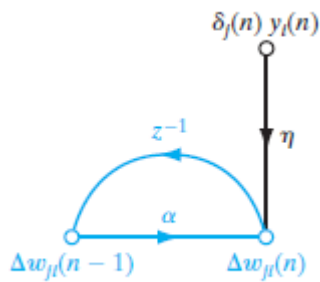


FIGURE 4.6 Signal-flow graph illustrating the effect of momentum constant  $\alpha$ , which lies inside the feedback loop.

In detail:

#### Rate of learning and momentum (time-series view)

The **generalized delta rule with momentum** is

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n) \quad (4.41)$$

Where  $\alpha$ : momentum constant  $\eta$ : learning rate.

If we solve this first-order recurrence (difference equation), we get

$$\Delta w_{ji}(n) = \eta \sum_{t=0}^n \alpha^{n-t} \delta_j(t) y_i(t) \quad (4.42-4.43)$$

Equivalently, using the gradient explicitly: 
$$\Delta w_{ji}(n) = -\eta \sum_{t=0}^n \alpha^{n-t} \frac{\partial e(t)}{\partial w_{ji}(t)} \quad (4.43)$$

Interpretation:

- If  $\frac{\partial e(t)}{\partial w_{ji}(t)}$  keeps the **same sign** over iterations  $\rightarrow$  the sum grows  $\rightarrow$  faster descent (accelerated learning).
- If it **changes sign** frequently  $\rightarrow$  the sum shrinks  $\rightarrow$  smoother changes and less oscillation.

Thus, momentum helps **speed up learning in stable directions** and **stabilize** oscillatory directions.

#### Stopping Criteria

Typical **convergence criteria** for back-propagation:

1. **Gradient norm small**
  - The Euclidean norm of the gradient vector becomes less than a threshold.
2. **Error change per epoch small**
  - Absolute change in average squared error per epoch is very small (e.g., 0.1–1% or even 0.01%).
3. **Generalization performance peaks**
  - After each learning iteration, test on validation set; stop when performance stops improving.



## Summary of the BP algorithm (algorithmic steps)

For each training example  $(x(n), d(n))$ :

### 1. Initialization

- Choose all weights and biases from a **uniform distribution** with mean 0 and suitable variance.

### 2. Forward computation

- For each layer  $l$ , compute

$$v_j^{(l)}(n) = \sum_i w_{ji}^{(l)}(n) y_i^{(l-1)}(n) \quad (4.44)$$
$$y_j^{(l)}(n) = \phi_j(v_j^{(l)}(n))$$

- In the input layer  $y_j^{(0)}(n) = x_j(n)$ .
- In the output layer, outputs are  $o_j(n) = y_j^{(L)}(n)$ , and errors

$$e_j(n) = d_j(n) - o_j(n) \quad (4.45)$$

### 3. Backward computation

- Compute local gradients  $\delta_j^{(L)}(n)$  and  $\delta_j^{(l)}(n)$  using (4.46).
- Update weights using (4.47).

### 4. Repeat for all training examples and multiple epochs until a **stopping criterion** is satisfied.

---

## 4. XOR Problem

**Text source:** Section 4.5, pages 141–143

The XOR function is **not linearly separable**, so a single perceptron cannot solve it.

### Solution using MLP

A network with **one hidden layer of 2 neurons** can solve XOR.

Key ideas:

- Hidden neurons form decision boundaries (Fig. 4.9).
- Output neuron combines them to produce the XOR function.

This illustrates how MLP handles **nonlinear classification**.

XOR problem in detail:

### Definition of XOR

Consider two binary inputs  $x_1$  and  $x_2$ .

The XOR function outputs 1 if **exactly one** of the inputs is 1, otherwise 0.

We can write:

$$\begin{aligned} 0 \oplus 0 &= 0 \\ 1 \oplus 1 &= 0 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \end{aligned}$$

So, among the four combinations:

- $(0,0) \rightarrow$  class 0
- $(1,1) \rightarrow$  class 0
- $(0,1) \rightarrow$  class 1
- $(1,0) \rightarrow$  class 1

Now imagine these four points as the **corners of a unit square** in the  $(x_1, x_2)$  plane:

- $(0,0)$  — bottom-left
- $(1,0)$  — bottom-right
- $(0,1)$  — top-left
- $(1,1)$  — top-right

Points in the same class lie at **opposite corners** of the square.

---

### 3. Why a single neuron cannot solve XOR

A single neuron with two inputs and a threshold activation gives a **single straight line** as the decision boundary in the input space.

- On one side of this line → output class 1
- On the other side → output class 0

For XOR, we would need a boundary that separates:

- (0,0) and (1,1) (class 0)
- from (0,1) and (1,0) (class 1)

But these points are arranged such that **no single straight line** can separate them in that way.

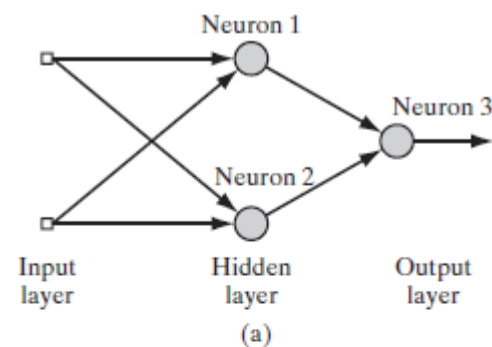
Therefore, **a single-layer perceptron cannot implement XOR.**

### 4. Using an MLP with a hidden layer

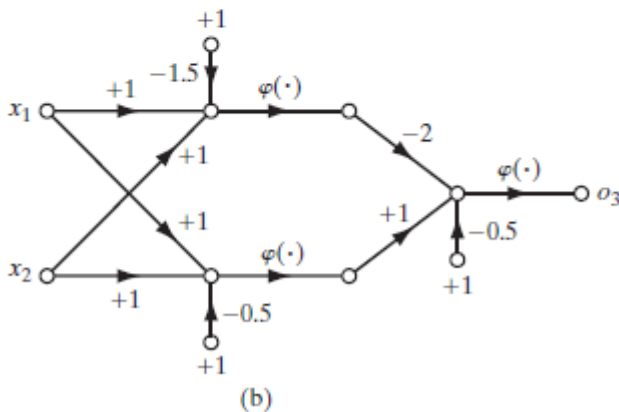
We can solve XOR by using a **multilayer perceptron** with:

- **Input layer:** 2 input nodes  $x_1, x_2$
- **Hidden layer:** 2 neurons (Neuron 1 and Neuron 2)
- **Output layer:** 1 neuron (Neuron 3)

This architecture is shown conceptually in Figure 4.8(a) in the textbook.



**FIGURE 4.8** (a) Architectural graph of network for solving the XOR problem. (b) Signal-flow graph of the network.



#### Assumptions

1. Each neuron is a **McCulloch–Pitts neuron** with a **threshold activation function** (i.e., outputs 0 or 1 depending on whether its input is below/above threshold).
2. The binary symbols **0** and **1** are represented by numeric levels (these are taken as **0** and **+1**, respectively).

#### Hidden layer neurons: Neuron 1 and Neuron 2

##### 5.1 Parameters of Neuron 1 (top hidden neuron)

Weights and bias for **Neuron 1**:

$$w_{11} = +1, w_{12} = +1, b_1 = -\frac{3}{2}$$

Its induced local field is:

$$v_1 = w_{11}x_1 + w_{12}x_2 + b_1 = x_1 + x_2 - \frac{3}{2}$$

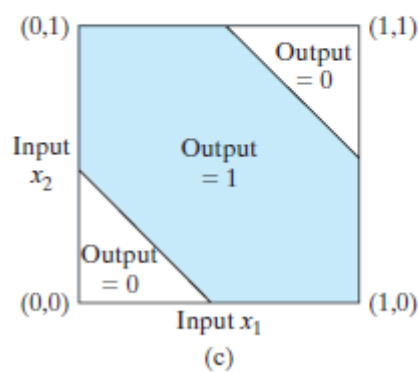
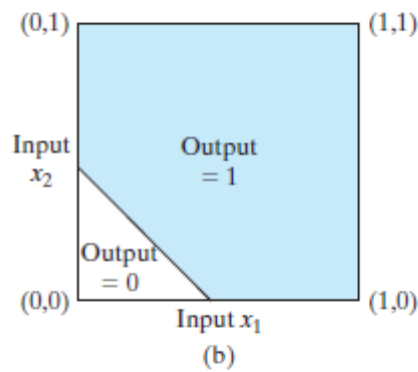
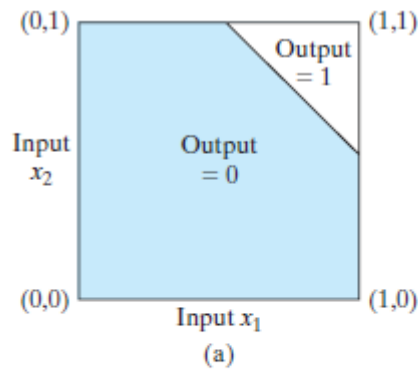
The decision boundary of Neuron 1 is obtained by setting  $v_1 = 0$ :

$$x_1 + x_2 - \frac{3}{2} = 0 \Rightarrow x_2 = -x_1 + \frac{3}{2}$$

So the decision line has **slope -1**.

The position of this line relative to the four input points is shown in Figure 4.9(a).

**FIGURE 4.9** (a) Decision boundary constructed by hidden neuron 1 of the network in Fig. 4.8.  
 (b) Decision boundary constructed by hidden neuron 2 of the network.  
 (c) Decision boundaries constructed by the complete network.



For each input pair:

1.  $(0,0): v_1 = 0 + 0 - \frac{3}{2} = -\frac{3}{2} < 0 \Rightarrow \text{Neuron 1 output} = 0$
2.  $(1,0): v_1 = 1 + 0 - \frac{3}{2} = -\frac{1}{2} < 0 \Rightarrow \text{output} = 0$
3.  $(0,1): v_1 = 0 + 1 - \frac{3}{2} = -\frac{1}{2} < 0 \Rightarrow \text{output} = 0$
4.  $(1,1): v_1 = 1 + 1 - \frac{3}{2} = \frac{1}{2} > 0 \Rightarrow \text{output} = 1$

So **Neuron 1** fires (output 1) **only** for input (1,1).

#### Parameters of Neuron 2 (bottom hidden neuron)

Weights and bias for **Neuron 2**:  $w_{21} = +1, w_{22} = +1, b_2 = -\frac{1}{2}$

Induced local field:  $v_2 = w_{21}x_1 + w_{22}x_2 + b_2 = x_1 + x_2 - \frac{1}{2}$

Decision boundary ( $v_2 = 0$ ):  $x_1 + x_2 - \frac{1}{2} = 0 \Rightarrow x_2 = -x_1 + \frac{1}{2}$

Again slope is **-1**, but the line is located differently (see Figure 4.9(b)).

For each input:

$$1. \quad (0,0): v_2 = 0 + 0 - \frac{1}{2} = -\frac{1}{2} < 0 \Rightarrow \text{Neuron 2 output} = 0$$

$$2. \quad (1,0): v_2 = 1 + 0 - \frac{1}{2} = \frac{1}{2} > 0 \Rightarrow \text{output} = 1$$

$$3. \quad (0,1): v_2 = 0 + 1 - \frac{1}{2} = \frac{1}{2} > 0 \Rightarrow \text{output} = 1$$

$$4. \quad (1,1): v_2 = 1 + 1 - \frac{1}{2} = \frac{3}{2} > 0 \Rightarrow \text{output} = 1$$

So **Neuron 2** fires (output 1) for **(1,0), (0,1), and (1,1)**, but not for (0,0).

### Output neuron: Neuron 3

The output neuron (Neuron 3) receives inputs from both hidden neurons.

Parameters:  $w_{31} = -2, w_{32} = +1, b_3 = -\frac{1}{2}$

Let  $h_1$  and  $h_2$  denote the outputs of Neuron 1 and Neuron 2 respectively.

Then the induced local field at Neuron 3 is:  $v_3 = w_{31}h_1 + w_{32}h_2 + b_3$

And its output is:

- 1 if  $v_3 > 0$
- 0 if  $v_3 \leq 0$

Now compute for each input  $(x_1, x_2)$ :

#### Case 1: Input (0,0)

From earlier:

- Neuron 1 output  $h_1 = 0$
- Neuron 2 output  $h_2 = 0$

So:

$$v_3 = (-2)(0) + (1)(0) - \frac{1}{2} = -\frac{1}{2} < 0 \Rightarrow \text{Output } o_3 = 0$$

Desired XOR output: 0 → **Correct.**

#### Case 2: Input (1,1)

- Neuron 1 output  $h_1 = 1$
- Neuron 2 output  $h_2 = 1$

Then:

$$v_3 = (-2)(1) + (1)(1) - \frac{1}{2} = -2 + 1 - \frac{1}{2} = -\frac{3}{2} < 0 \Rightarrow o_3 = 0$$

Desired XOR output: 0 → **Correct.**

#### Case 3: Input (0,1)

- Neuron 1 output  $h_1 = 0$
- Neuron 2 output  $h_2 = 1$

Then:

$$v_3 = (-2)(0) + (1)(1) - \frac{1}{2} = 1 - \frac{1}{2} = \frac{1}{2} > 0 \Rightarrow o_3 = 1$$

Desired XOR output: 1 → **Correct.**

#### Case 4: Input (1,0)

- Neuron 1 output  $h_1 = 0$
- Neuron 2 output  $h_2 = 1$  (same as case 3 due to symmetry)

$$v_3 = (-2)(0) + (1)(1) - \frac{1}{2} = \frac{1}{2} > 0 \Rightarrow o_3 = 1$$

Desired XOR output: 1 → **Correct**.

So this **three-layer network with 2 hidden neurons** correctly implements the XOR function

### Geometric interpretation (using the figures)

- **Neuron 1** defines a line (Fig. 4.9(a)) that **separates (1,1)** from the other three points.
- **Neuron 2** defines another line (Fig. 4.9(b)) that **separates (0,0)** from the other three points.
- The **output neuron** then **combines** these two decision boundaries to produce the final partition shown in **Figure 4.9(c)**:
  - Class 1 region contains **(0,1)** and **(1,0)**.
  - Class 0 region contains **(0,0)** and **(1,1)**.

Thus, the MLP realizes a **nonlinear decision boundary** by combining multiple linear boundaries created by hidden neurons.

---

## 5. Heuristics to Improve Backpropagation Performance

**Text source:** Section 4.6, pages 144–149

### 1. Stochastic (online) learning leads to faster convergence

Shuffling examples avoids redundancy.

There are two ways to update weights:

#### (a) Batch Update (Non-sequential)

- The gradient is computed **after processing all training examples**.
- One weight update per epoch.
- Accurate gradient but **computationally expensive**, especially when:
  - The dataset is **large**
  - The data is **highly redundant**
- Reason: computing the **Jacobian matrix** becomes expensive for redundant data.

#### (b) Stochastic Update (Sequential / Pattern-by-pattern)

- Update weights **after each training example**.
- Much **faster** in practice.
- Especially good for:
  - Redundant datasets
  - Large datasets

### Conclusion:

Stochastic BP is **preferred** because it converges faster and avoids the Jacobian computation problems of batch mode.

### 2. Maximizing Information Content in Training Examples

Training examples should be selected to provide **maximum information**.

Two ways to choose informative examples:

#### (i) Use examples that produce the largest training error

These examples:

- Provide the strongest corrective signal.
- Encourage the network to explore weight space more effectively.

#### (ii) Use examples that are “radically different” from those used earlier

- This avoids repeatedly training on similar samples.
- Helps cover **more regions** of input space.

### Shuffling in each epoch

In classification tasks using sequential BP:

- Shuffle the order of examples in each epoch.
- Avoids consecutive patterns from the **same class**.
- Prevents local learning biases.

### 3. Choice of Activation Function

Learning is faster if the activation function:

#### (i) Is an **odd** function of its argument

A function is odd if:

$$\phi(-v) = -\phi(v)$$

Examples:

- Hyperbolic tangent:  $\phi(v) = \tanh(bv)$  — **odd**
- Logistic sigmoid: **not odd**

Thus, **hyperbolic tangent** is preferred.

#### (ii) Slope near 1 at the origin

This ensures adequate sensitivity of neurons during training.

#### Recommended activation function:

From LeCun (1993):

$$\phi(v) = \tanh(bv)$$

with:

$$a = 1.7159, b = \frac{2}{3}$$

Properties:

- $\phi(1) \approx 1, \phi(-1) \approx -1$
- $\phi'(0) = ab \approx 1.1424$
- Second derivative maximum at  $v = 1$

These values give the neuron:

- Good dynamic range
- Sensitivity near the origin
- Faster learning

### 4. Choice of Target Values

Target values must lie **within the range** of the activation function.

#### Why?

If targets are too close to saturation limits (e.g.,  $\pm 1.7159$  for  $\tanh$ ):

- Back-propagation pushes weights to **extremely large values**.
- Causes **hidden neuron saturation**.
- Slows learning dramatically.

#### For **tanh** activation:

$$\phi(v) = \tanh(bv), a = 1.7159$$

Therefore, instead of using  $\pm a$  as targets, choose:

$$\begin{aligned} d_j &= a - \epsilon \\ d_j &= -a + \epsilon \end{aligned}$$

where  $\epsilon$  is a positive constant.

The book suggests:  $\epsilon = 0.7159$

So the target values become:  $d_j = \pm 1$

Thus, recommended target levels: **+1 and -1**.

These lie in the **non-saturated region** and accelerate learning.

### 5. Normalizing the Inputs

Input preprocessing is **critical** for faster and stable learning.

#### (a) Mean should be close to zero

If inputs are always positive:

- All weights connected to first layer will move together (increase or decrease).
- Weight vector can only change direction by **zigzagging**, slowing learning.

#### (b) Inputs should be decorrelated

Use **Principal Component Analysis (PCA)** to:

- Remove correlation between inputs
- Rotate the space so inputs vary independently

### (c) Inputs should have similar variance

Equalizing variances:

- Ensures all weights learn at a similar rate.

Normalization prevents slow, unstable convergence.

## 6. Initializing the Weights Properly

Good initialization avoids:

- Saturated neurons
- Tiny gradients
- Slow learning

**For tanh activation with parameters  $a$  and  $b$ :**

Goal:

$$\text{Var}(v_j) = 1$$

Derivation (from book):

$$v_j = \sum_{i=1}^m w_{ji} y_i$$
$$\text{Var}(v_j) = \sum_{i=1}^m \text{Var}(w_{ji}) \text{Var}(y_i)$$

Setting desired variance:

$$\text{Var}(w_{ji}) = \frac{1}{m}$$

Thus:

**Recommended rule:**

Select initial weights from a **uniform distribution** with:

- Mean = **0**
- Variance =  **$1/m$** , where  $m$  = number of synaptic connections to the neuron.

## 7. Learning from Hints

Learning normally uses only training examples.

But we can enhance it by adding **prior knowledge** about the function  $f(\cdot)$ :

Examples of hints:

- Invariances
- Symmetries
- Constraints
- Structural properties of the data

Hints guide the search through weight space more efficiently.

Equation (4.48) in the text is given as an example of incorporating prior knowledge.

---

## 8. Learning Rates (Assigning $\eta$ Properly)

All neurons should ideally learn at the **same speed**.

But:

- Neurons in **later layers** typically have **larger local gradients**.
- Neurons with **more inputs** change more slowly.

**Therefore:**

**(a) Use smaller learning rates for deeper layers**

→ Prevents overshooting.

(b) Neurons with many inputs should have smaller  $\eta$

This ensures uniform learning speed across the network.

**Recommended rule (LeCun, 1993):**

$$\eta \propto \frac{1}{\sqrt{m}}$$

where

- $m$  = number of synaptic connections into the neuron.

#### 4.8 BACK PROPAGATION AND DIFFERENTIATION

Back-propagation is a **specific method** for implementing **gradient descent** in the weight space of a multilayer perceptron. Its goal is to compute **partial derivatives** of the approximating function:

$$F(w, x)$$

with respect to **all adjustable weights** in the vector  $w$ , for a given input vector  $x$ .

**Why is this important?**

Because training a neural network requires updating all weights in the direction that **reduces the error**, and this requires knowing:

$$\frac{\partial F(w, x)}{\partial w_{ji}^{(l)}}$$

for every weight in every layer.

This section explains how BP computes these derivatives **efficiently**, using the **chain rule** and the structure of the network.

**Network Architecture (from Fig. 4.14)**

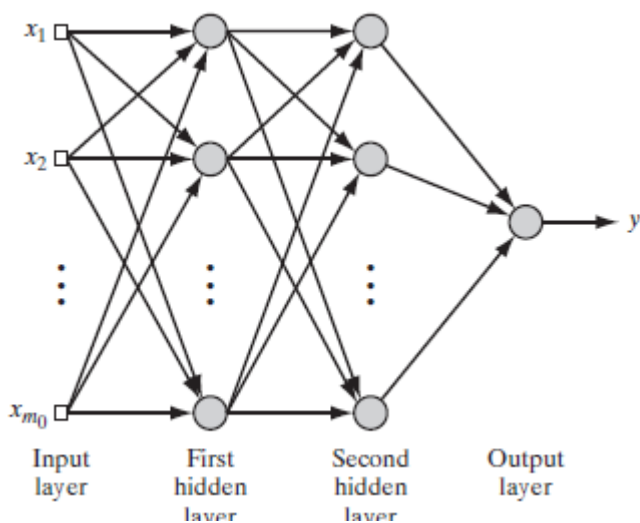


FIGURE 4.14 Multilayer perceptron with two hidden layers and one output neuron.

A multilayer perceptron with:

- An **input layer** with  $m_0$  nodes,
- Two **hidden layers**,
- **One output neuron**.

Weights are organized so that:

- $w_{ji}^{(l)}$  = synaptic weight from neuron  $i$  in layer  $l - 1$  to neuron  $j$  in layer  $l$ .
- For layer  $l = 1$ , input layer  $\rightarrow$  hidden layer, index  $i$  refers to an **input component**, not a neuron.

The full weight vector  $w$  is ordered layer-wise.

#### Expressing the Network Function

For a simple case (one hidden layer + linear output):

$$F(w, x) = \sum_{j=0}^{m_1} w_{0j}^{(2)} \phi\left(\sum_{i=0}^{m_0} w_{ji}^{(1)} x_i\right) \quad (4.49)$$

General form (3-layer model in Fig. 4.14):



$$F(w, x) = \phi(a_1^{(3)}) \quad (4.50)$$

where

- $a_j^{(l)}$  represents the activation argument for neuron  $j$  in layer  $l$ .
- $\phi(\cdot)$  is the activation function.

### Partial Derivatives Used for Backpropagation

Using the chain rule, and adapting earlier formulas (from Sections 4.2–4.4), we derive:

**Derivative of  $F$  w.r.t. a weight in output layer (layer 3):**

$$\frac{\partial F(w, x)}{\partial w_{1k}^{(3)}} = \phi'(a_1^{(3)}) \phi(a_k^{(2)}) \quad (4.51)$$

**Derivative w.r.t. a weight in second hidden layer (layer 2):**

$$\frac{\partial F(w, x)}{\partial w_{kj}^{(2)}} = \phi'(a_1^{(3)}) \phi'(a_k^{(2)}) \phi(a_j^{(1)}) w_{1k}^{(3)} \quad (4.52)$$

**Derivative w.r.t. a weight in first hidden layer (layer 1):**

$$\frac{\partial F(w, x)}{\partial w_{ji}^{(1)}} = \phi'(a_1^{(3)}) \phi'(a_k^{(2)}) \phi'(a_j^{(1)}) w_{1k}^{(3)} w_{kj}^{(2)} x_i \quad (4.53)$$

These equations show that **gradients are computed by moving backward layer by layer**, multiplying:

1. Derivatives of activation functions
2. Weights
3. Earlier activations

Which is exactly the mechanism of backpropagation.

### Sensitivity of the Network Function

Let  $\#$  be a particular weight from the vector  $w$ .

The **sensitivity** of the network output to that weight is:

$$S_{\#} = \frac{\partial F(w, x)}{\partial \#}$$

This sensitivity determines the **direction and amount** of weight adjustment during training.

The lower part of the signal-flow diagram (Fig. 4.7 in Chapter 4) is therefore called the **sensitivity graph**.

### Jacobian Matrix

For a network with:

- $N$  training examples
- $W$  total adjustable weights

We compute  **$W$  partial derivatives** for each example:

$$\frac{\partial F(w, x(n))}{\partial w_1}, \frac{\partial F(w, x(n))}{\partial w_2}, \dots, \frac{\partial F(w, x(n))}{\partial w_W}$$

Repeating for all  $n = 1 \dots N$ , we get an  $N \times W$  matrix:

$$J = \begin{bmatrix} \frac{\partial F(w, x(1))}{\partial w_1} & \dots & \frac{\partial F(w, x(1))}{\partial w_W} \\ \vdots & \ddots & \vdots \\ \frac{\partial F(w, x(N))}{\partial w_1} & \dots & \frac{\partial F(w, x(N))}{\partial w_W} \end{bmatrix}$$

This matrix is called the **Jacobian**.

### Why is the Jacobian important?

- It affects how the BP algorithm searches in weight space.
- If the Jacobian is **rank-deficient**, the training becomes:

- **Slow**
- **Less stable**
- Unable to explore all directions of weight adjustments

Many neural network problems are **intrinsically ill-conditioned**, which means their Jacobian tends to be nearly rank-deficient.